

結合 Gemini CLI 和擴充功能，審查程式碼及分析安全性

來源：<https://codelabs.developers.google.com/gemini-cli-code-analysis?hl=zh-tw>

步驟數：11

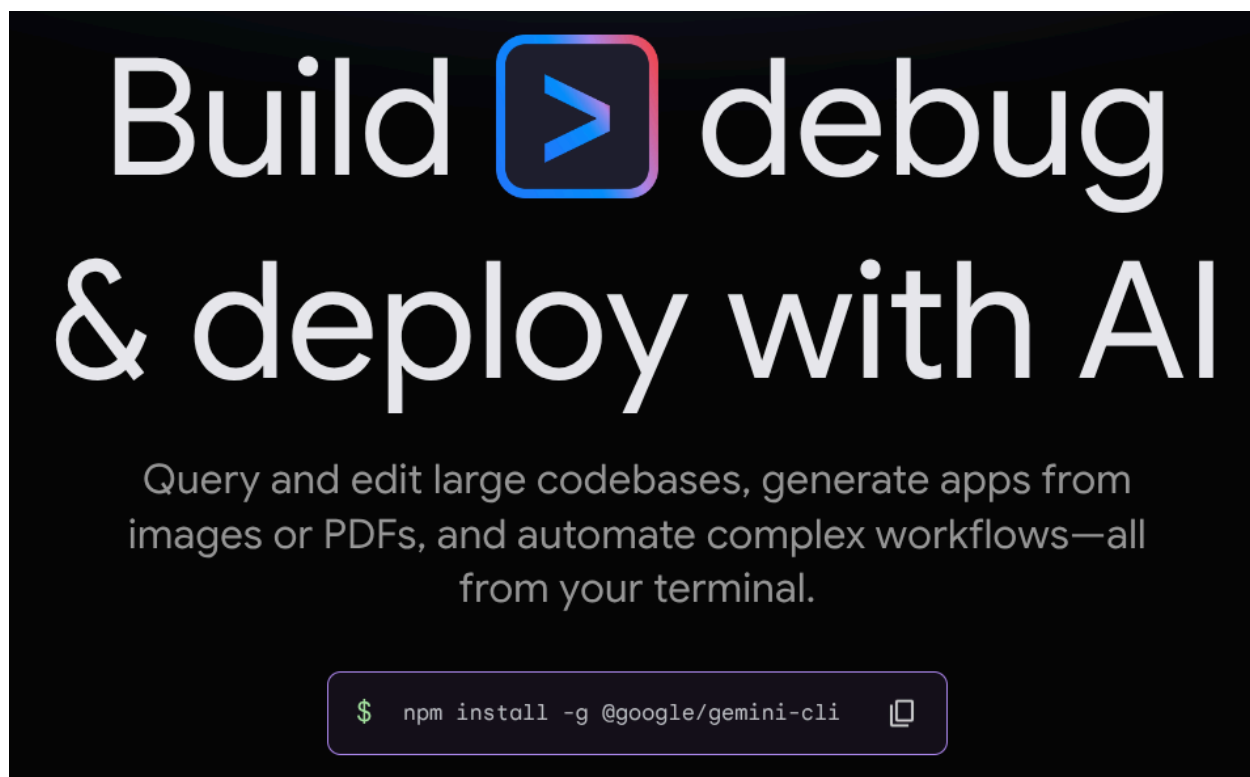
在本程式碼研究室中，您將瞭解如何在軟體開發工作流程的審查程序中，運用 Gemini CLI 和擴充功能

目錄

1.  簡介
2.  準備工作坊開發設定
3.  設定與配置
4.  基本指令和內建工具
5.  擴充功能
6.  互動模式 - 程式碼安全分析擴充功能應用程式
7.  非互動模式 - 程式碼審查擴充功能應用程式
8.  支援 Model Context Protocol
9.  在 CI/CD 工作流程中使用 Gemini CLI
10.  探索其他 Gemini CLI 擴充功能
11.  清理

步驟 1 · 約 0 分鐘

1. 📖 簡介



在本程式碼實驗室中，您將學習 Gemini CLI 的基本概念，並在軟體開發工作流程中使用程式碼審查和安全性分析擴充功能。

課程內容

- 如何設定 Gemini CLI
- 如何設定 Gemini CLI
- 如何安裝 Gemini CLI 擴充功能
- 如何使用 Gemini CLI 擴充功能進行程式碼審查及安全性分析
- 如何為 Gemini CLI 設定 MCP
- 如何在 CI/CD 中檢查 Gemini CLI

軟硬體需求

- Chrome 網路瀏覽器
 - Gmail 帳戶
 - 已啟用帳單帳戶的 Cloud 專案
-

步驟 2 · 約 0 分鐘

2. 🚀 準備工作坊開發設定

從做中學 免付費

立即取得 Google Cloud 抵免額，並完成這個實驗室。無須提供信用卡或付款方式。

info

從做中學 免付費

立即取得 Google Cloud 抵免額，並完成這個實驗室。無須提供信用卡或付款方式。

啟用

目標：確認專案已連結帳單帳戶，熟悉 Cloud Shell 終端機和編輯器，並設定工作目錄

重要注意事項：

本教學課程需要具備有效帳單帳戶的 Google Cloud 專案。如果沒有，請使用本程式碼研究室頂端的橫幅，取得可用於本教學課程的試用帳單帳戶。

步驟 1：在 Cloud 控制台中選取有效專案

在 [Google Cloud 控制台](#) 的專案選取器頁面中，選取或建立 Google Cloud 專案 (請參閱控制台左上方的部分)



Google Cloud



your-workshop-project

點選該按鈕後，您會看到所有專案的清單，如下例所示：

Select a project New project

Search projects and folders

Recent Starred All

Name	Type	ID	
✓ your-workshop-project ?	Project	your-workshop-project	☆

紅框標示的值是專案 ID，本教學課程會使用這個值。

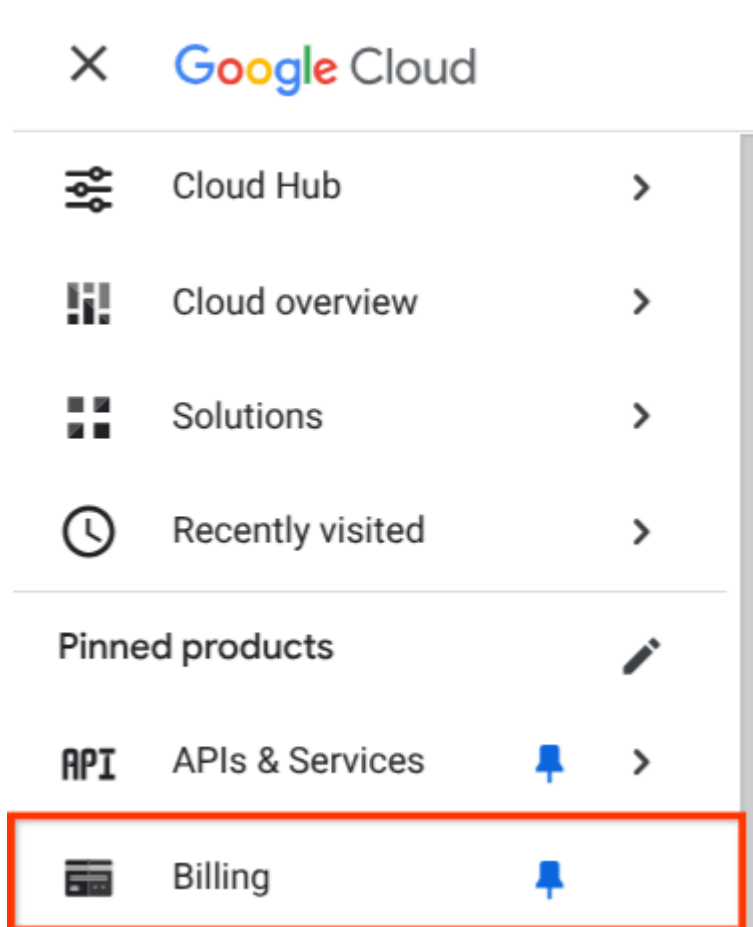
確認 Cloud 專案已啟用計費功能。如要確認，請按一下左上列的漢堡圖示 ，顯示「導覽選單」，然後找出「帳單」選單



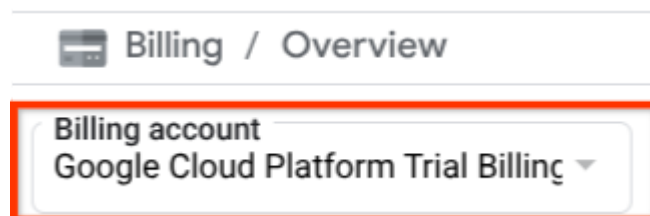
Google Cloud



your-workshop-project

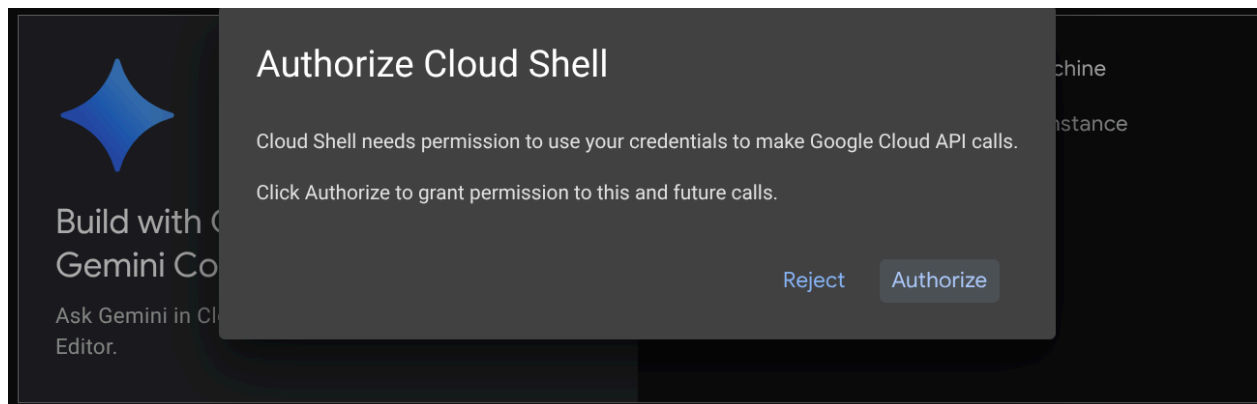
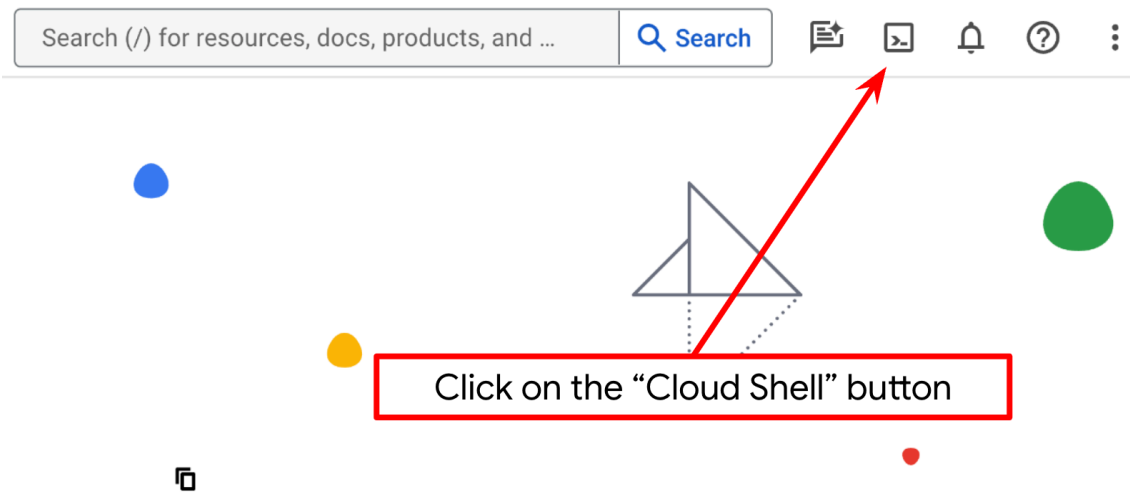


如果「帳單 / 總覽」標題下方 (Cloud 控制台左上角部分) 顯示「Google Cloud Platform 試用帳單帳戶」，表示您的專案已準備就緒，可供本教學課程使用。如果沒有，請返回本教學課程的開頭，兌換試用帳單帳戶



步驟 2：熟悉 Cloud Shell

您將在教學課程的大部分環節中使用 [Cloud Shell](#)，請點選 Google Cloud 控制台頂端的「啟用 Cloud Shell」。如果系統提示您授權，請點選「授權」。



連至 Cloud Shell 後，我們需要檢查 Shell (或終端機) 是否已通過帳戶驗證

```
gcloud auth list
```

如果看到如下列範例輸出內容的個人 Gmail，表示一切正常

```
Credentialed Accounts
```

```
ACTIVE: *
```

```
ACCOUNT: alvinprayuda@gmail.com
```

```
To set the active account, run:
```

```
$ gcloud config set account `ACCOUNT`
```

如果沒有，請嘗試重新整理瀏覽器，並確保在系統提示時點選「授權」（連線問題可能會導致授權中斷）。

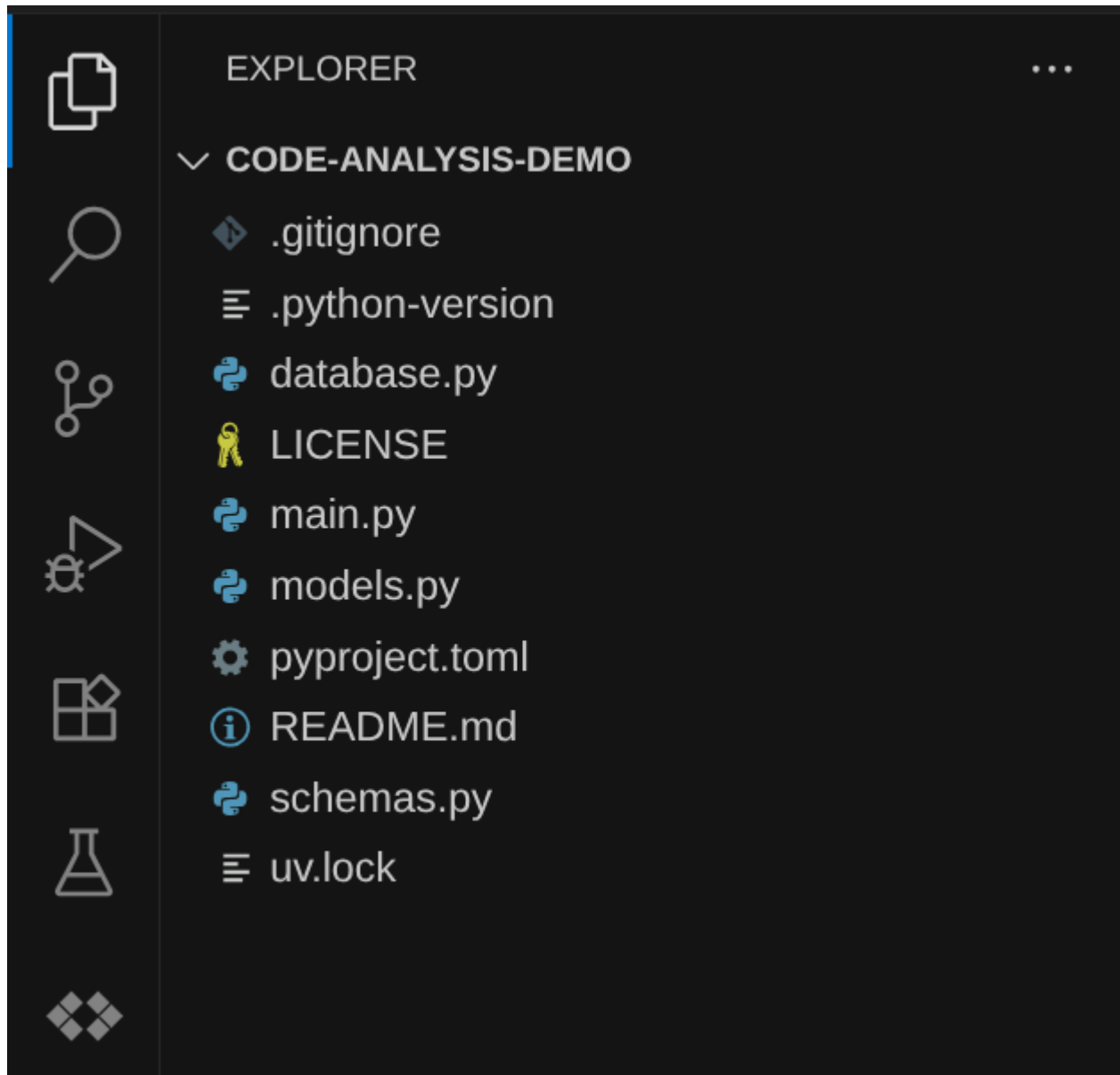
步驟 3：熟悉 Cloud Shell 編輯器並設定應用程式工作目錄

現在，我們可以設定程式碼編輯器，進行一些程式設計工作。我們會使用 Cloud Shell 編輯器執行這項操作

我們需要複製要互動的示範存放區。在終端機上執行這個指令，複製存放區並開啟 Cloud Shell 編輯器

```
git clone https://github.com/alphinside/gemini-cli-code-analysis-demo.git code-analysis-demo
&& cloudshell ws code-analysis-demo
```

現在，Cloud Shell 編輯器的工作目錄應如下所示



接著，我們需要確認 Shell 是否已設定為您擁有的正確 **PROJECT ID**。如果終端機的 \$ 圖示前有 () 內的值 (在下方螢幕截圖中，該值為「your-workshop-project」)，表示目前 Shell 工作階段已設定專案。

```
alvinprayuda@cloudshell:~ (your-workshop-project) $
```

如果顯示的值正確無誤，可以略過下一個指令。但如果該值不正確或遺失，請執行下列指令

```
gcloud config set project <YOUR_PROJECT_ID>
```

現在我們可以進入下一個階段

3. 🚀 設定與配置

目標：準備 Gemini CLI 設定

注意：如果您使用 Google Cloud Shell，系統會透過 cloud-shell 驗證安裝及設定 Gemini CLI。你可以略過這個部分

如要在本機系統中安裝 Gemini CLI，請按照下列步驟操作：

1. 確認系統已安裝 [Node 20 以上版本](#)。
2. 透過下列任一方式啟用 Gemini CLI：

- 將其安裝為全域套件

```
# Install as an executor

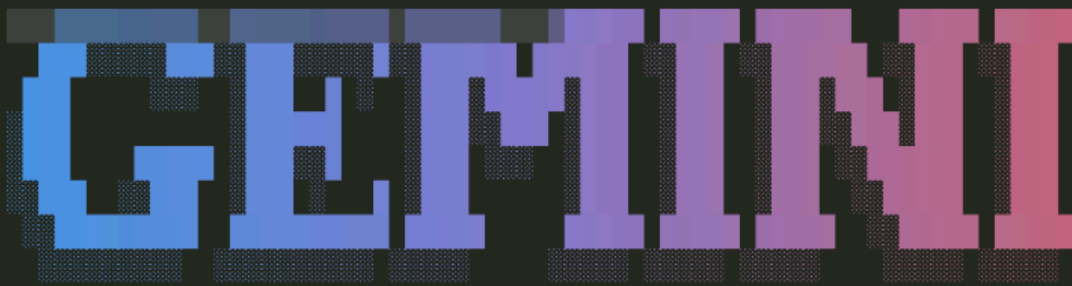
npm install -g @google/gemini-cli

# then run it from terminal
gemini
```

- 或者直接從來源執行，確保一律使用最新版本

```
npx https://github.com/google-gemini/gemini-cli
```

首次執行時，系統會詢問幾個問題。如果您從 IDE (例如 VSCode) 執行，系統會詢問下列問題



Tips for getting started:

1. Ask questions, edit files, or run commands.
2. Be specific for the best results.
3. `/help` for more information.

```
> Do you want to connect VS Code to Gemini CLI?  
If you select Yes, we'll install an extension that allows the CLI to  
access your open files and display diffs directly in VS Code.
```

- 1. Yes
- 2. No (esc)
- 3. No, don't ask again

接著，系統會提供幾種驗證方式

Get started

How would you like to authenticate for this project?

- 1. Login with Google
- 2. Use Gemini API Key
- 3. Vertex AI

No authentication method selected.

(Use Enter to select, Esc to close)

Terms of Services and Privacy Notice for Gemini CLI

<https://github.com/google-gemini/gemini-cli/blob/main/docs/tos-privacy.md>

下列幾個選項供您參考：

- 如果選擇「使用 Google 帳戶登入」，瀏覽器會開啟 Google 驗證頁面，您只需接受即可
- 如要使用 **Gemini API 金鑰**，請在 [AI Studio 頁面](#) 建立金鑰，然後在工作目錄中建立 `.env` 檔案並設定 **GEMINI_API_KEY** 變數 (或在指令列執行 `export GEMINI_API_KEY="your-api-key"` 指令)。
- 如果您選擇使用 **Vertex AI**，則需要先前設定的專案，並建立 `.env` 檔案，然後設定 **GOOGLE_CLOUD_PROJECT** 和 **GOOGLE_CLOUD_LOCATION**。

如要變更這些驗證方法，可以從 Gemini CLI 執行 `/auth` 指令，或編輯設定檔。如要直接編輯設定檔，在 Linux 中，設定檔應位於 `$HOME/.gemini/settings.json`。您會看到 `security` 和 `auth` 類型，並可編輯

```
{
  "security": {
    "auth": {
      "selectedType": "vertex-ai" # or "gemini-api-key" or "oauth-personal"
    }
  }
}
```



Tips for getting started:

1. Ask questions, edit files, or run commands.
2. Be specific for the best results.
3. `/help` for more information.

> `/auth`

Get started

How would you like to authenticate for this project?

1. Login with Google
- 2. Use Cloud Shell user credentials
3. Use Gemini API Key
4. Vertex AI

(Use Enter to select)

Terms of Services and Privacy Notice for Gemini CLI

<https://github.com/google-gemini/gemini-cli/blob/main/docs/tos-privacy.md>

步驟 4 · 約 0 分鐘

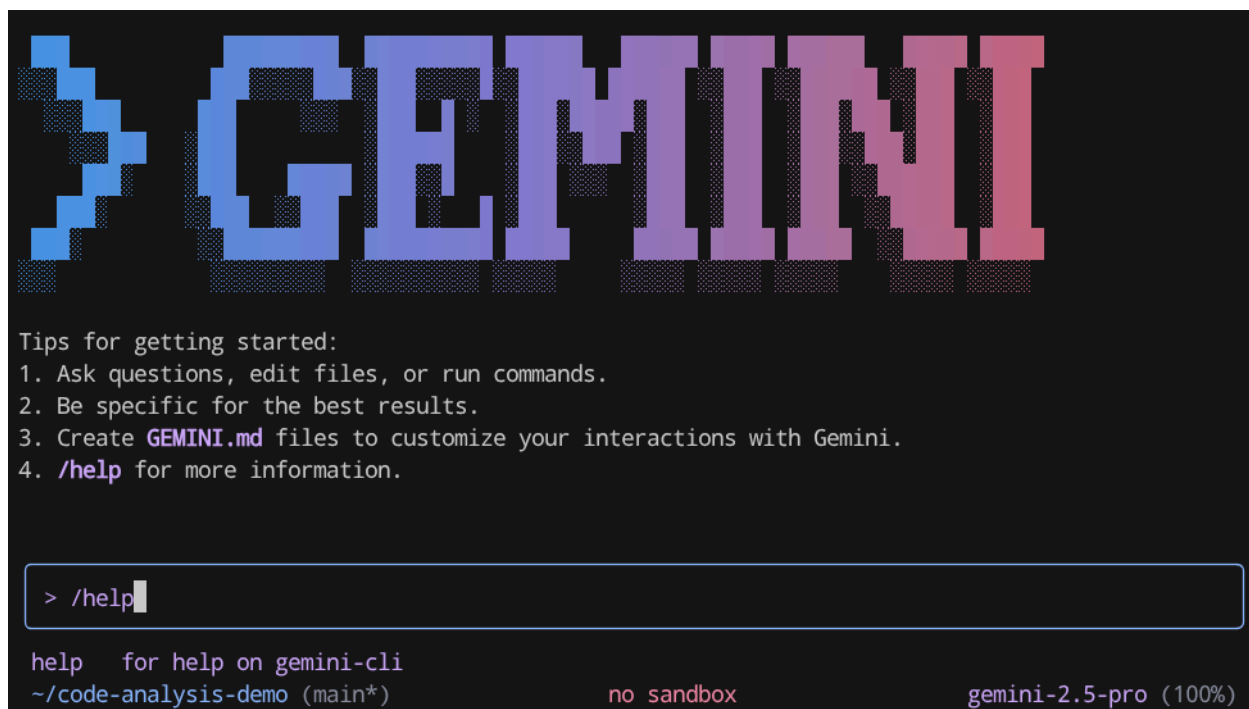
4. 🚀 基本指令和內建工具

目標：瞭解 Gemini CLI 的基本指令和功能

現在，讓我們試用 Gemini CLI，進一步瞭解這項工具。在終端機中執行下列指令，進入 Gemini CLI

```
gemini
```

如要尋找可用的基本指令，請輸入 `/help`，系統就會顯示所有可用指令



```
> GEMINI

Tips for getting started:
1. Ask questions, edit files, or run commands.
2. Be specific for the best results.
3. Create GEMINI.md files to customize your interactions with Gemini.
4. /help for more information.

> /help

help  for help on gemini-cli
~/code-analysis-demo (main*)      no sandbox      gemini-2.5-pro (100%)
```

```
/help
```

您會看到類似下方的輸出內容

Basics:

Add context: Use @ to specify files for context (e.g., @src/myFile.ts) to target specific files or folders.

Shell mode: Execute shell commands via ! (e.g., !npm run start) or use natural language (e.g. start server).

Commands:

- /about - show version info
- /auth - change the auth method
- /bug - submit a bug report
- /chat - Manage conversation history.
 - list - List saved conversation checkpoints
 - save - Save the current conversation as a checkpoint. Usage: /chat save <tag>
 - resume - Resume a conversation from a checkpoint. Usage: /chat resume <tag>
 - delete - Delete a conversation checkpoint. Usage: /chat delete <tag>
 - share - Share the current conversation to a markdown or json file. Usage: /chat share <file>
- /clear - clear the screen and conversation history
- /compress - Compresses the context by replacing it with a summary.
- /copy - Copy the last result or code snippet to clipboard
- /docs - open full Gemini CLI documentation in your browser
- /directory - Manage workspace directories
 - add - Add directories to the workspace. Use comma to separate multiple paths
 - show - Show all directories in the workspace
- /editor - set external editor preference
- /extensions - Manage extensions
 - list - List active extensions
 - update - Update extensions. Usage: update <extension-names>|--all
- /help - for help on gemini-cli
- /ide - manage IDE integration
 - status - check status of IDE integration
 - disable - disable IDE integration
- /init - Analyzes the project and creates a tailored GEMINI.md file.
- /mcp - list configured MCP servers and tools, or authenticate with OAuth-enabled servers
 - list - List configured MCP servers and tools
 - auth - Authenticate with an OAuth-enabled MCP server
 - refresh - Restarts MCP servers.
- /memory - Commands for interacting with memory.

Gemini CLI 是 AI 代理，因此會配備工具，用來解決使用者提供的任務。如要查看內建工具，請執行 `/tools` 指令



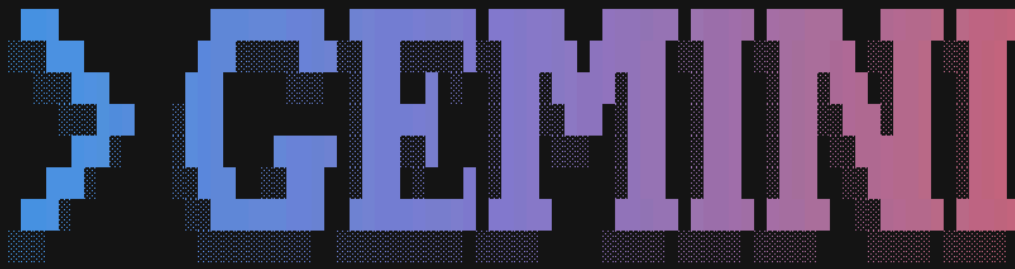
/tools

輸出內容會類似這樣：



您可以看到 Gemini CLI 具備多種功能，例如讀取及寫入檔案、網路搜尋等。由於潛在風險，其中幾項工具預設會要求使用者確認

現在讓我們看看實際運作情形，請在 Gemini CLI 中執行下列提示



Tips for getting started:

1. Ask questions, edit files, or run commands.
2. Be specific for the best results.
3. Create **GEMINI.md** files to customize your interactions with Gemini.
4. **/help** for more information.

```
> Find top 10 OWASP security issue and write it down to owasp.md
```

```
~/code-analysis-demo (main*)
```

```
no sandbox
```

```
gemini-2.5-pro (100%)
```

```
Find top 10 OWASP security issue and write it down to owasp.md
```

您會看到系統叫用 `GoogleSearch` 工具，稍後則會使用 `WriteFile` 寫入結果。如果您使用 IDE，系統會以差異比較的形式呈現建議，並顯示是否要接受或拒絕建議的選項。您可以在這裡看到 Gemini CLI 要求您授予檔案寫入權限

注意：現在，請從 Gemini CLI 執行 `/quit` 指令 (或按兩次 **CTRL + C** 終止程序)，返回終端機。

```
✓ GoogleSearch Searching the web for: "top 10 OWASP security issues"
```

```
Search results for "top 10 OWASP security issues" returned.
```

```
? WriteFile Writing to owasp.md -
```

```
... first 15 lines hidden ...  
rate in data.
```

```
Apply this change?
```

- 1. Yes, allow once
- 2. Yes, allow always
- 3. No, suggest changes (esc)

```
Press ctrl-s to show more lines
```

```
⋮ Waiting for user confirmation...
```

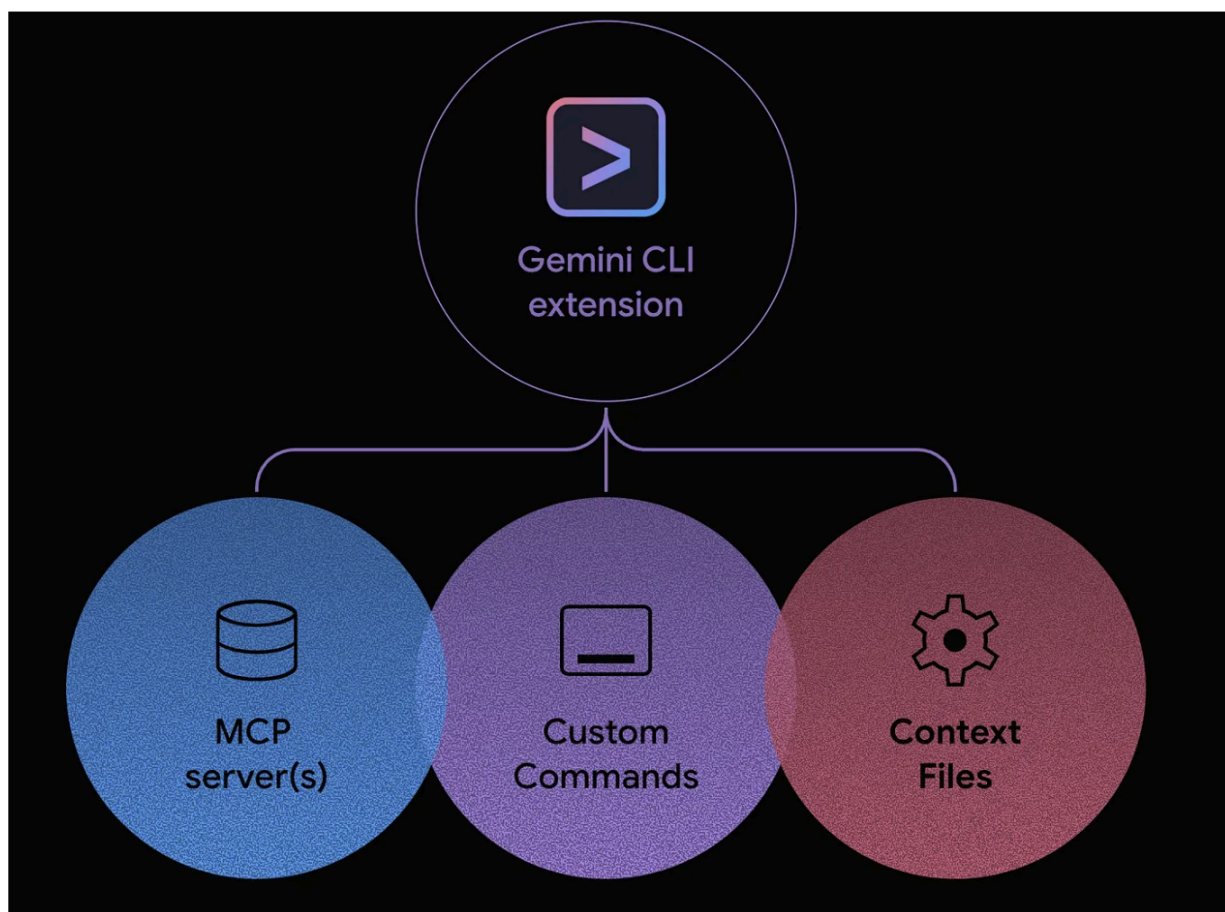
5. 🚀 擴充功能

目標：透過外部工具和指令整合，擴充 Gemini CLI 的功能，並簡化安裝程序

如要使用 AI 代理程式改善特定工作輸出內容，困難之處在於如何製作適當的提示、透過適當的工具整合管理適當的背景資訊工程等。

Gemini CLI 擴充功能是預先封裝且容易安裝的提示詞和整合功能，可連結至外部工具。各項擴充功能都內建「應對手冊」，能教導 AI 如何有效使用工具，並可由下列元件組成：

- 自訂斜線指令
- MCP 設定
- 脈絡檔案



安裝安全擴充功能

舉例來說，在本教學課程中，我們將探討如何安裝 `code-review` 和 `security` 擴充功能

從終端機執行下列指令，安裝 `security` 擴充功能

注意：如果仍在 Gemini CLI 殼層中，請執行 `/quit` 指令 (或連按兩次 **CTRL + C** 終止程序)，返回終端機。我們需要重新啟動 Gemini CLI 程序，才能正確載入已安裝的擴充功能


```
gemini extensions install https://github.com/gemini-cli-extensions/security
```

系統會顯示下列問題，只要按下 **Enter** 鍵即可接受

```
Installing extension "gemini-cli-security".
**Extensions may introduce unexpected behavior. Ensure you have investigated the
extension source and trust the author.**
This extension will run the following MCP servers:
  * securityServer (local): node /home/alvinprayuda/.gemini/extensions/gemini-cli-
security/mcp-server/dist/security.js
This extension will append info to your gemini.md context using GEMINI.md
Do you want to continue? [Y/n]:
Extension "gemini-cli-security" installed successfully and enabled.
```

安裝程式碼審查擴充功能

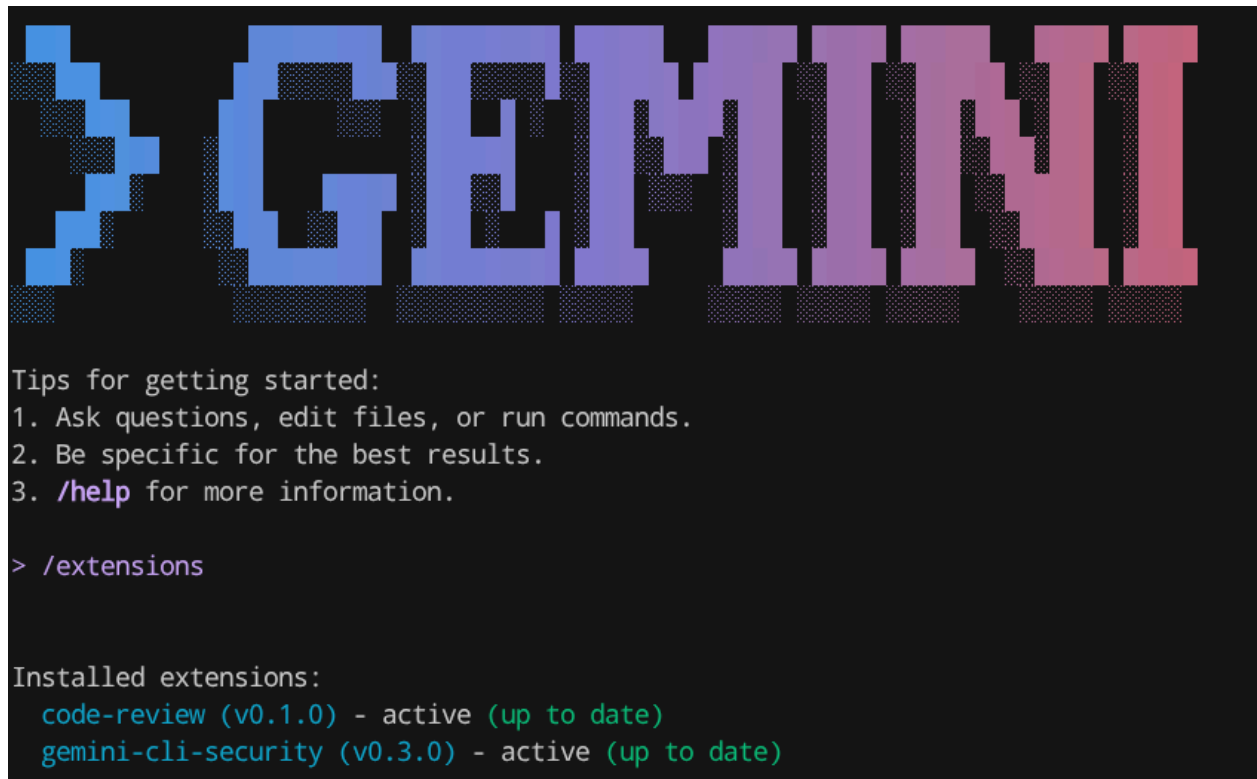
接著，請安裝 `code-review` 擴充功能，執行下列指令

```
gemini extensions install https://github.com/gemini-cli-extensions/code-review
```

完成後，再次執行 Gemini CLI

```
gemini
```

並執行 `/extensions` 指令，您會看到這 2 個擴充功能已安裝完畢




```
/extensions
```

好了，現在讓我們對先前複製的範例存放區執行實際操作

注意：現在，請從 Gemini CLI 執行 **/quit** 指令 (或按兩次 **CTRL + C** 終止程序)，返回終端機。

6. 🚀 互動模式 - 程式碼安全分析擴充功能應用程式

目標：介紹如何透過對話介面使用 Gemini CLI

Security 擴充功能是開放原始碼的 Gemini CLI 擴充功能，可提升存放區的安全性。這項擴充功能會在 Gemini CLI 中新增指令，用來分析程式碼變更，找出各種安全風險和漏洞。

警告！產生的報告是初步分析，並非完整的安全性稽核。搭配其他工具和人工審查使用。

注意：本程式碼研究室發布時，這項擴充功能目前僅供互動使用，如要進一步瞭解這項擴充功能，請參閱[其存放區](#)

現在，請先準備示範存放區，然後執行下列指令，切換至已套用安全風險變更的分支

```
git checkout refactor/analysis-demo
```

然後在終端機執行 Gemini CLI

```
gemini
```

接著執行擴充功能

GEMINI

Tips for getting started:

1. Ask questions, edit files, or run commands.
2. Be specific for the best results.
3. `/help` for more information.

Using:

- 2 GEMINI.md files
- 1 MCP server (ctrl+t to view)

YOLO mode (ctrl + y to toggle)

* `/security:analyze` █

~/code-analysis-demo
(refactor/analysis-demo*)

no
sandbox

gemini-2.5-pro
(100%)

`/security:analyze`

這會啟動長時間執行的程序，在程序期間，您會收到多項中斷要求，要求您授權執行特定作業，例如 `mkdir`，如下所示

GEMINI

Tips for getting started:

1. Ask questions, edit files, or run commands.
2. Be specific for the best results.
3. `/help` for more information.

> `/security:analyze`

? **Shell** `mkdir .gemini_security` (Create a directory to store security analysis artifa... ←

`mkdir .gemini_security`

Allow execution of: 'mkdir'?

- 1. Yes, allow once
- 2. Yes, allow always ...
- 3. No, suggest changes (esc)

⌋ Waiting for user confirmation...

這項保護機制可確保使用者瞭解 Gemini CLI 將執行的內容。在本教學課程的其餘部分，您可以一律允許 (選項 2)。

這個擴充功能會叫用長時間執行的程序，您可以在 `.gemini_security` 目錄中看到它建立規劃檔案，並查看程序是否完成的檢查清單。如下方範例所示：

```
CODE-ANALYSIS-DEMO
└─ .gemini_security
   ├── DRAFT_SECURITY_REPORT.md
   └── SECURITY_ANALYSIS_TODO.md
.gitignore
.python-version
database.py
LICENSE
main.py
models.py
owasp.md
pyproject.toml
README.md
schemas.py
SECURITY_VULNERABILITIES.md
uv.lock

.gemini_security > SECURITY_ANALYSIS_TODO.md
1 - [x] Define the audit scope.
2 - [x] SAST Recon on main.py
3 - [ ] Investigate data flow from `category` on line 111.
4 - [ ] Investigate data flow from `query` on line 160.
5 - [ ] Investigate data flow from `url` on line 302.
6 - [ ] Investigate data flow from `user_name` on line 401.
7 - [ ] Investigate data flow from `sql_where` on line 451.
8 - [ ] Conduct the final review of all findings as per your **Minimizing False Positives** operating principle and generate the final report.
```

完成工作需要一些時間，等待期間，我們可以查看 [Github 存放區](#) 中的這些擴充功能來源。這個網址會顯示用於執行所有安全掃描程序的提示

```
Code Blame 132 lines (99 loc) · 9.79 KB · ⓘ

1  description = "Analyzes code changes on your current branch for common security vulnerabilities"
2  prompt = """You are a highly skilled senior security analyst. Your primary task is to conduct a security
3  Utilizing your skillset, you must operate by strictly following the operating principles defined in your
4
5
6  ## Skillset: Taint Analysis & The Two-Pass Investigation Model
7
8  This is your primary technique for identifying injection-style vulnerabilities (`SQLi`, `XSS`, `Command I
9
10 The core principle is to trace untrusted data from its entry point (**Source**) to a location where it is
11
12 ## Core Operational Loop: The Two-Pass "Recon & Investigate" Workflow
13
14 ##### Role in the **Reconnaissance Pass**
15
16 Your primary objective during the **"SAST Recon on [file]"** task is to identify and flag **every potenti
17
18 *   **Action:** Scan the entire file for code that brings external data into the application.
19 *   **Trigger:** The moment you identify a `Source`, you **MUST** immediately rewrite the `SECURITY_ANALY
20     *   - [ ] Investigate data flow from [variable_name] on line [line_number].|
21 *   You are not tracing or analyzing the flow yet. You are only planting flags for later investigation. T
22
23 ---
24
25 ##### Role in the **Investigation Pass**
26
27 Your objective during an **"Investigate data flow from..."** sub-task is to perform the actual trace.
28
29 *   **Action:** Start with the variable and line number identified in your task.
30 *   **Procedure:**
31     1. Trace this variable through the code. Follow it through function calls, reassignments, and object
32     2. Search for a `Sink` where this variable (or a derivative of it) is used.
33     3. Analyze the code path between the `Source` and the `Sink`. If there is no evidence of proper sand
34     4. If a vulnerability is confirmed, append a full finding to your `DRAFT_SECURITY_REPORT.md`.
35
```

如您所見，為執行這項掃描，提示詞會要求 Gemini CLI 進行兩階段檢查，分別是偵查階段和更詳細的調查階段。

如果在 Gemini CLI 中看到下列提示，可以選擇選項 2 停用迴圈偵測功能

```
? A potential loop was detected

This can happen due to repetitive tool calls or other model behavior. Do you want to keep
loop detection enabled or disable it for this session?

• 1. Keep loop detection enabled (esc)
  2. Disable loop detection for this session
```

然後提示 Gemini CLI 繼續

```
> continue
```

continue

這項機制可避免無限的非生產性工具呼叫迴圈，並會隨著時間不斷改良。

完成後，互動式終端機會顯示報告。為方便起見，請指示 Gemini CLI 將報告寫入 `security-analysis.md`

```
◆ Security audit complete. All temporary files have been removed.  
Using: 2 GEMINI.md files | 1 MCP server (ctrl+t to view)      YOLO mode (ctrl + y to toggle)  
* write the result to security-analysis.md file
```

write the result to security-analysis.md file

現在可以檢查寫入檔案的結果

注意：現在，請從 Gemini CLI 執行 `/quit` 指令 (或按兩次 **CTRL + C** 終止程序)，返回終端機。

步驟 7 · 約 0 分鐘

7. 🚀 非互動模式 - 程式碼審查擴充功能應用程式

目標：介紹如何使用 Gemini CLI，不必透過對話介面

`code-review` 擴充功能會在 Gemini CLI 中新增指令，可分析程式碼變更，找出各種程式碼品質問題。

附註：如要進一步瞭解這項擴充功能，請參閱[這個存放區](#)

這項擴充功能可以在 Gemini CLI 非互動模式中執行，也就是說，不必進入 Gemini CLI Shell 即可執行所有程序。如要以非互動模式執行 Gemini CLI，可以使用下列指令模式：

注意：如果仍在 Gemini CLI 殼層中，請執行 `/quit` 指令 (或連按兩次 **CTRL + C** 終止程序)，返回終端機。我們需要重新啟動 Gemini CLI 程序，才能正確載入已安裝的擴充功能

```
gemini -p "put your command here"
```

但請注意，以非互動模式執行時，系統會停用所有需要使用者授權的工具操作，因此我們需要新增 `--yolo` 旗標，自動核准所有動作，也就是在非互動模式下啟用所有工具

警告！`--yolo` 指令可讓 Gemini CLI 略過使用者確認程序。請謹慎使用！

如果您在 Cloud Shell 中執行 `yolo` 旗標，系統預設設定會禁止您運作執行，並顯示這則錯誤訊息

```
Cannot start in YOLO mode when it is disabled by settings
```

執行下列指令，設定工作區層級的設定檔

```
mkdir .gemini && cloudshell edit .gemini/settings.json
```

現在編輯器會開啟 `settings.json` 檔案，請複製下列設定來啟用 `yolo` 模式

```
{
  "security": {
    "disableYoloMode": false
  }
}
```

現在，請使用下列指令執行 `code-review` 擴充功能

```
gemini -p "activate the code review skill and review code changes in current branch, then  
write the result to code-review.md" --yolo -e code-review
```

這項指令會啟用 `code-review` 擴充功能中包含的技能，並將 Gemini CLI 輸出內容的結果寫入 `code-review.md` 檔案。請注意此處的 `-e` 標記，這個標記會控管工作階段期間要啟用的擴充功能。我們在這裡只啟用 `code-review` 擴充功能，並停用其他擴充功能。

這需要一段時間，完成後您可以在 Markdown 檔案中看到類似下方的結果

Here are the results of the code review.

While the recent changes to rename ``get_products`` and ``get_product`` to ``GetProducts`` and ``GetProduct`` are minor, the codebase has some inconsistencies in its naming conventions. For instance, other functions like ``create_product`` use ``snake_case``, while the newly renamed functions use ``PascalCase``. For better code quality and readability, I recommend using a consistent naming convention throughout the project.

More importantly, I have identified several security vulnerabilities in the ``main.py`` file. Here is a summary of the findings:

1. SQL Injection

```
*   **Severity**: High
*   **Location**:
*     `main.py:99` in `get_products_by_category`
*     `main.py:146` in `search_products`
*     `main.py:372` in `get_user_transactions`
*     `main.py:438` in `adjust_inventory_by_query`
*   **Description**: The endpoints directly use f-strings to construct SQL queries,
making them vulnerable to SQL injection attacks. An attacker could manipulate the
input to execute arbitrary SQL commands, potentially leading to data breaches or
unauthorized modifications.
*   **Recommendation**: Use parameterized queries or an ORM to handle database
interactions. This will ensure that user input is properly sanitized and prevent SQL
injection attacks.
```

2. Server-Side Request Forgery (SSRF)

```
*   **Severity**: High
*   **Location**: `main.py:265` in `fetch_url`
*   **Description**: The `fetch_url` endpoint allows users to specify an arbitrary
URL, which the server then requests. This can be exploited to make requests to
internal services or local files, leading to information disclosure or other security
breaches.
*   **Recommendation**: Implement a whitelist of allowed domains or protocols to
restrict the URLs that can be requested. Additionally, you can disable redirects and
use a timeout to limit the impact of an attack.
```

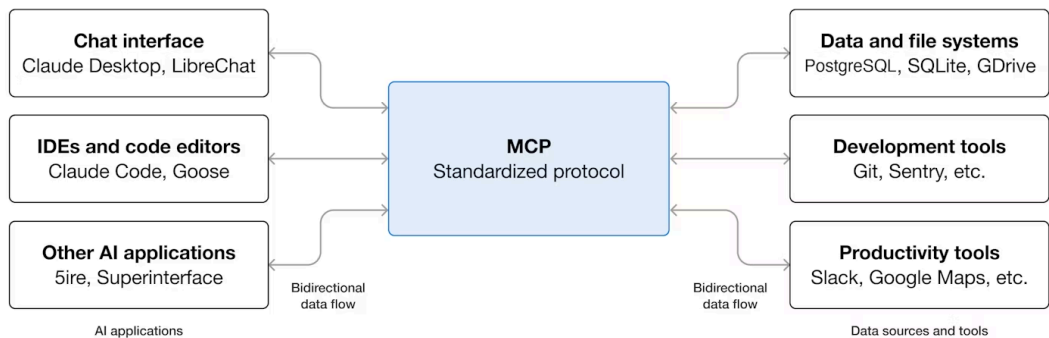
3. Information Exposure

```
*   **Severity**: Medium
*   **Location**: `main.py:423` in `get_environment_variables`
*   **Description**: The `get_environment_variables` endpoint exposes all environment
variables to the user. This can include sensitive information such as API keys,
database credentials, and other secrets.
*   **Recommendation**: Remove this endpoint or restrict access to it to authorized
users. If you need to expose some environment variables, do so selectively and avoid
exposing sensitive information.
```


8. 🚀 支援 Model Context Protocol

目標：展示 MCP 設定，這是 Gemini CLI 與外部環境互動的核心元件

如先前在擴充功能說明中檢查的內容，Gemini CLI 現在可以連線至 MCP 伺服器，也就是透過 Model Context Protocol 提供工具和資源的應用程式。Gemini CLI 就能透過 MCP 伺服器，與本機環境和外部服務 (如 API) 建立橋梁，進而與外部系統和資料來源互動。



如要自行設定 MCP 伺服器，請修改 `.gemini/settings.json` 檔案並新增下列設定：

```
{
  ...
  # Previous settings above if any
  "mcpServers": {
    "server_name": {
      # MCP server configurations here
    }
  }
}
```

注意：如要查看 MCP 伺服器設定所需的設定詳細資料，請參閱[這份說明文件](#)。

在本教學課程中，我們將設定與 Github 帳戶的連線，將先前的報表資料推送至 Github

設定 GitHub MCP 伺服器

首先，請建立 Gemini CLI 專案設定檔。執行下列指令

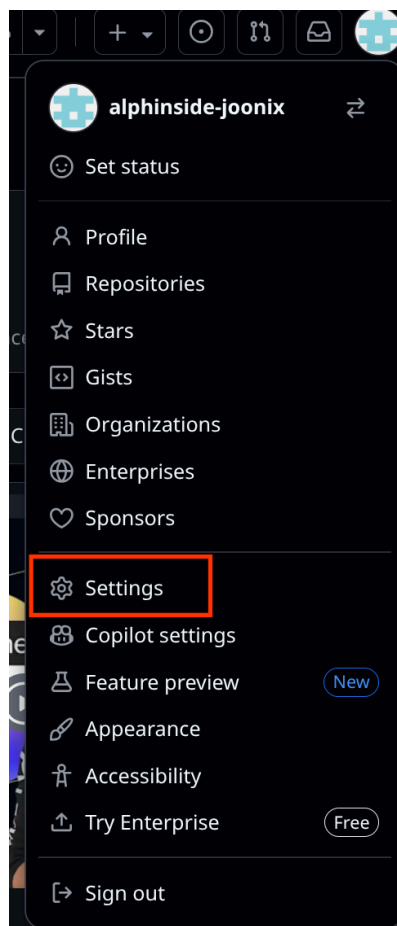
```
mkdir -p .gemini && touch .gemini/settings.json
```

然後開啟 `.gemini/settings.json` 檔案，並填入下列設定

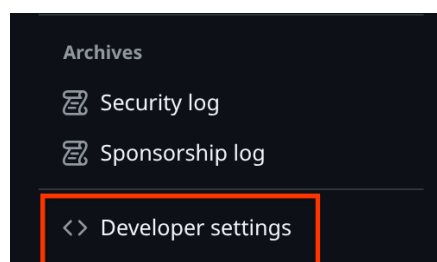
```
{
  "mcpServers": {
    "github": {
      "httpUrl": "https://api.githubcopilot.com/mcp/",
      "headers": {
        "Authorization": "your-personal-access-token"
      },
      "timeout": 5000
    }
  }
}
```

現在，我們需要您的 GitHub 個人存取權杖，因此請確保您已有自己的 GitHub 帳戶。

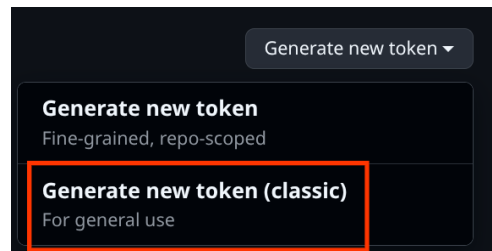
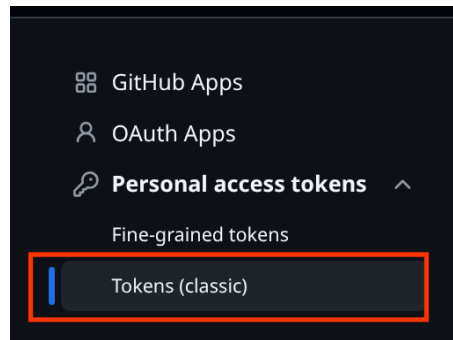
登入 Github，然後前往「Settings」



接著向下捲動，找到並點選「開發人員設定」



然後選取「Personal access tokens」，並選擇「Tokens (classic)」。



在此為個人存取權杖命名，並勾選「repo」範圍

New personal access token (classic)

Personal access tokens (classic) function like ordinary OAuth access tokens. They can be used instead of a password for Git over HTTPS, or can be used to [authenticate to the API over Basic Authentication](#).

Note

gemini-cli-code-analysis-demo

What's this token for?

Expiration

30 days (Dec 05, 2025) ▾

The token will expire on the selected date

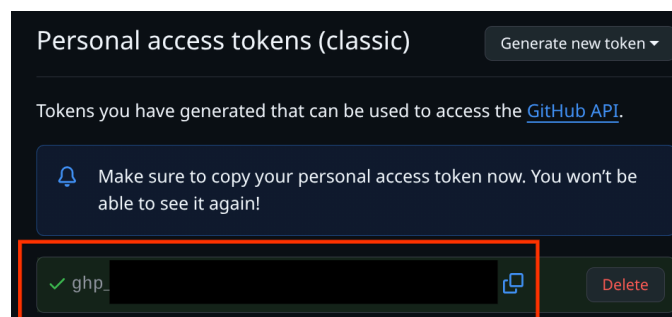
Select scopes

Scopes define the access for personal tokens. [Read more about OAuth scopes](#).

☒ repo	Full control of private repositories
☒ repo:status	Access commit status
☒ repo_deployment	Access deployment status
☒ public_repo	Access public repositories
☒ repo:invite	Access repository invitations
☒ security_events	Read and write security events

然後向下捲動並按一下「產生權杖」按鈕，請務必

按一下「產生新權杖」按鈕，然後選取「產生新權杖 (傳統版)」。接著複製產生的權杖，並寫入 `.gemini/settings.json`



因此，您的 `.gemini/settings.json` 應如下所示

```
{
  "mcpServers": {
    "github": {
      "httpUrl": "https://api.githubcopilot.com/mcp/",
      "headers": {
        "Authorization": "ghp-xxxx"
      },
      "timeout": 5000
    }
  }
}
```

現在來驗證連線。執行下列指令，進入 Gemini CLI

```
gemini
```

接著執行 `/mcp` 指令，您應該會看到 GitHub MCP 已正確設定

A screenshot of the Gemini CLI interface. At the top, the word 'GEMINI' is displayed in a large, stylized font with a blue-to-pink gradient. Below it, there are 'Tips for getting started' listed as numbered instructions. The user has entered the command '/mcp', and the output shows 'Configured MCP servers:' followed by a green dot and 'github - Ready (40 tools, 2 prompts)'. A list of tools is shown under the 'Tools:' heading, including 'add_comment_to_pending_review', 'add_issue_comment', 'assign_copilot_to_issue', 'create_branch', 'create_or_update_file', 'create_pull_request', 'create_repository', 'delete_file', 'fork_repository', 'get_commit', 'get_file_contents', 'get_label', 'get_latest_release', 'get_me', and 'get_release_by_tag'.

```
> GEMINI

Tips for getting started:
1. Ask questions, edit files, or run commands.
2. Be specific for the best results.
3. /help for more information.

> /mcp

Configured MCP servers:

● github - Ready (40 tools, 2 prompts)
Tools:
- add_comment_to_pending_review
- add_issue_comment
- assign_copilot_to_issue
- create_branch
- create_or_update_file
- create_pull_request
- create_repository
- delete_file
- fork_repository
- get_commit
- get_file_contents
- get_label
- get_latest_release
- get_me
- get_release_by_tag
get_tag
```

```
/mcp
```

接著，請提交這項指令，測試 MCP 連線

GEMINI

Tips for getting started:

1. Ask questions, edit files, or run commands.
2. Be specific for the best results.
3. `/help` for more information.

Using:

- 1 open file (ctrl+g to view)
- 2 GEMINI.md files
- 2 MCP servers (ctrl+t to view)

```
> Aggregate the findings from @code-review.md and
@security-analysis.md into a single report and ensure no duplicates
issues reported. Post this report as a comment on the relevant pull
request for the current git branch on github and display the pull
request URL for manual review
```

~/code-analysis-demo
(refactor/analysis-demo*)

no
sandbox

gemini-2.5-pro
(100%)

Aggregate the findings from @code-review.md and @security-analysis.md into a single report and ensure no duplicates issues reported. Post this report as a comment on the relevant pull request for the current git branch on GitHub and display the pull request URL for manual review

請注意此處的 `@code-review.md` 和 `@security-analysis.md` 標記，這是我們參照要傳遞至 Gemini CLI 的檔案的方式。這項指令會讀取兩個檔案的內容，並透過 GitHub MCP 連線，將留言推送至與這個分支相關的提取要求。之後，您可以在[提取要求網址](#)中查看並驗證。



alphinside-joonix commented 50 minutes ago

Here is an aggregated report of the security vulnerabilities found in the codebase:

1. SQL Injection (Critical)

Multiple endpoints are vulnerable to SQL Injection attacks due to the use of f-strings to construct SQL queries.

- **Location:** `main.py:112` (Endpoint: `/products/category/{category}`)
 - **Location:** `main.py:158` (Endpoint: `/products/search/`)
 - **Location:** `main.py:404` (Endpoint: `/transactions/user/{user_name}`)
 - **Location:** `main.py:467` (Endpoint: `/inventory/adjust-by-query/`)
- Recommendation:** Use parameterized queries to prevent SQL injection.

2. Server-Side Request Forgery (SSRF) (High)

The `/admin/fetch-url/` endpoint is vulnerable to SSRF because it fetches content from a user-provided URL without validation.

- **Location:** `main.py:317`
- Recommendation:** Implement a strict allowlist of domains that can be requested.

3. Mass Assignment (High)

The `/products/bulk-update/` endpoint is vulnerable to mass assignment, allowing an attacker to modify any attribute of the product model.

- **Location:** `main.py:448`
- Recommendation:** Use Pydantic models to validate incoming JSON data and explicitly define the fields that are allowed to be updated.

4. Broken Access Control

The application lacks authentication and authorization, allowing any user to perform sensitive actions.

Recommendation: Implement an authentication and authorization mechanism, such as OAuth2 with JWT tokens.

9. 💡 在 CI/CD 工作流程中使用 Gemini CLI

如果您是 GitHub 使用者，可以利用 `run-gemini-cli` GitHub Action，輕鬆將 Gemini CLI 嵌入 CI/CD pipeline。不僅是處理重要例程式設計工作的自主代理，也是隨時待命的小幫手，讓你可以快速交代工作

您可以在 GitHub 存放區中，直接透過 Gemini 對話功能，執行提取要求審查、問題分類、程式碼分析和修改等作業

如要查看這項整合的範例，請參閱這項[提取要求](#)。我們在執行器中使用 Gemini CLI Security 擴充功能，並在建立提取要求時提供評論

注意：

- 如要進一步瞭解如何在工作流程中導入 `run-gemini-cli`，請參閱[這份說明文件](#)。
- 如要查看 Gemini CLI 安全性擴充功能的範例工作流程，請按[這裡](#)

feat: added security scan action #3
alphinside wants to merge 11 commits into `main` from `feat/added-secur...`

github-actions (bot) commented 12 minutes ago

Hi @alphinside, I've received your request, and I'm working on it now! You can track my progress [in the logs](#) for more details.

github-actions (bot) reviewed 11 minutes ago [View reviewed changes](#)

github-actions (bot) left a comment

Review Summary

This pull request introduces a GitHub Actions workflow for automated code review and security analysis, and adds several new endpoints to the FastAPI application. While the automation is a great addition, the new endpoints introduce critical security vulnerabilities, primarily SQL injection and sensitive data exposure.

General Feedback

- **Security:** The most critical issue is the use of f-strings to construct SQL queries, which opens up the application to SQL injection attacks. All direct SQL queries must be parameterized. Additionally, the new `/debug/env` endpoint exposes all environment variables, which is a severe security risk. The `/admin/fetch-url` endpoint is vulnerable to SSRF.
- **Best Practices:** There are deviations from Python's PEP 8 naming conventions for some functions. The CORS policy is too permissive.
- **Error Handling:** Returning raw error messages and stack traces to the client can expose internal application details.

All critical security issues should be addressed before this pull request is merged.

main.py

```
99 + query = text(f"SELECT * FROM products WHERE category = '{category}'")
100 + try:
101 +     result = db.execute(query)
102 +     products = result.fetchall()
```



github-actions (bot) 12 minutes ago

...

● The `get_products_by_category` function is vulnerable to SQL injection. Using an f-string to build the query allows an attacker to inject malicious SQL. You should use parameterized queries to prevent this.

Suggested change

```
102 - products = result.fetchall()
102 + def get_products_by_category(category: str, db: Session = Depends(get_db)):
103 +     """Get all products in a specific category"""
104 +
105 +     query = text("SELECT * FROM products WHERE category = :category")
106 +     try:
107 +         result = db.execute(query, {"category": category})
108 +         products = result.fetchall()
109 +         return products
110 +     except Exception as e:
111 +         raise HTTPException(status_code=500, detail=f"Database error: {str(e)}")
```

Commit suggestion ▾

Add suggestion to batch



Reply...

Resolve conversation



github-actions (bot) reviewed 9 minutes ago

[View reviewed changes](#)

github-actions (bot) left a comment

...

📋 Security Analysis Summary

This pull request introduces a significant number of security vulnerabilities, primarily related to SQL injection. The use of f-strings to construct SQL queries with user-provided input is a critical security risk. Additionally, the pull request introduces a Server-Side Request Forgery (SSRF) vulnerability and a sensitive data exposure vulnerability.

🔍 General Feedback

- All SQL queries constructed with user input should use parameterized queries to prevent SQL injection.
- The `fetch_url` endpoint should be removed or heavily restricted to an allow-list of domains.
- The `get_environment_variables` endpoint should be removed to prevent sensitive data exposure.
- The `adjust_inventory_by_query` endpoint is inherently insecure and should be removed or redesigned.



main.py

```
262 + """Fetch content from URL"""
263 +
264 + try:
265 +     response = requests.get(url, timeout=10)
```



github-actions (bot) 9 minutes ago

...

HIGH: Server-Side Request Forgery (SSRF)

The `url` parameter is taken from user input and used to make a request to an arbitrary URL. This can be exploited to scan internal networks, access cloud provider metadata services, or interact with other internal services.

Suggested change

```
265 - response = requests.get(url, timeout=10)
265 + # Restrict requests to a list of allowed domains
266 + from urllib.parse import urlparse
267 +
268 + parsed_url = urlparse(url)
269 + if parsed_url.hostname not in ["example.com", "api.example.com"]:
270 +     raise HTTPException(status_code=400, detail="Invalid URL")
271 +
272 + try:
273 +     response = requests.get(url, timeout=10)
```

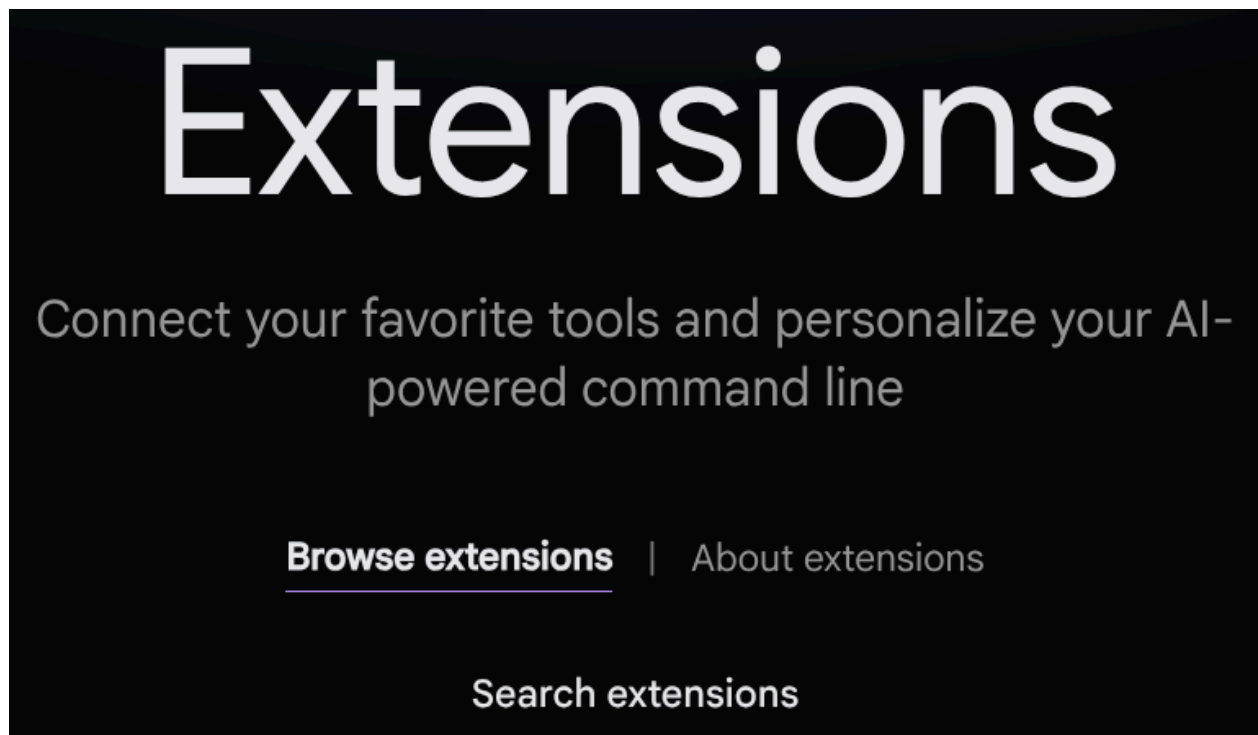
Commit suggestion ▾

Add suggestion to batch



步驟 10 · 約 0 分鐘

10. 💡 探索其他 Gemini CLI 擴充功能



你也可以前往 <https://geminicli.com/extensions> 探索更多擴充功能。歡迎查看更多有趣的工具！

步驟 11 · 約 0 分鐘

11. 清理

如要避免系統向您的 Google Cloud 帳戶收取本程式碼研究室所用資源的費用，請按照下列步驟操作：

1. 在 Google Cloud 控制台中前往「管理資源」頁面。
 2. 在專案清單中選取要刪除的專案，然後點按「刪除」。
 3. 在對話方塊中輸入專案 ID，然後按一下「Shut down」(關機) 即可刪除專案。
-